Using Modules to reuse and organize code

A module is a namespace that contains definitions of functions or types, and you can choose whether those definitions are visible outside their module (public) or not (private).

Here's an overview of how modules work:

- The 'mod' keyword declares a new module. Code within the module appears either immediately following this declaration within curly brackets or in another file.
- By default, functions, types, constants, and modules are private. The 'pub' keyword makes an item public and therefore visible outside its namespace.
- The 'use' keyword brings modules, or the definitions inside modules, into scope so it's easier to refer to them.

mod and the Filesystem

We'll start our example by making a new project with Cargo using '–lib' which creates a library crate: a project that other people can pull into their projects as a dependency.

We'll create a skeleton of a library that provides some general networking functionality; we'll concentrate on the organization of the modules and functions, but we won't worry about what code goes in the function bodies.

In this case we created the 'communicator/' folder.

Module definitions

We'll first define a module named network that contains the definition of a function called connect. Every module definition in Rust starts with the 'mod' keyword.

After the 'mod' keyword, we put the name of the module, network, and then a block of code in curly brackets. Everything inside this block is inside the namespace network. In this case, we have a single function named 'connect'.

We can also have multiple modules, side by side, in the same 'src/lib.rs' file.

In our example now we have 'network::connect()' and 'client::connect()' which could have completely different functionalities, and names do not conflict because they're in two different modules.

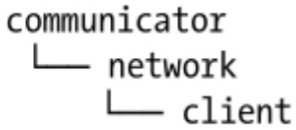
We can also put modules inside of modules, which can be useful as your modules grow to keep related functionality organized together and separate functionality apart.

The client code and its 'connect' function might make more sense to users of our library if they were inside the network namespace instead.

```
mod network {
    fn connect() {
    }

    mod client {
        fn connect() {
        }
    }
}
```

The functions 'network::connect()' and 'network::client::connect()' are both named connect, but still they don't conflict with each other because they're in different namespaces.



Moving modules to other files

We can use Rust's module system along with multiple files to split up Rust projects so not everything lives in `src/lib.rs` or `src/main.rs`.

For example we might want to to separate the 'client', 'network', and 'server' modules from `src/lib.rs` and place them into their own files.

We're still declaring the client module inside the `src/lib.rs`, but by replacing the block with a semicolon, we're telling Rust to look in another location for the code.

So we've now moved the content of the 'client' and 'network' modules content into their own files;

We still have to extract the 'server' module into its own file because it was a submodule of 'network'.

The error says we cannot declare a new module at this location and is pointing to the 'mod server;' line in `src/network.rs`.

Instead of continuing to follow the same file-naming pattern we used previously, we can do what the note suggests:

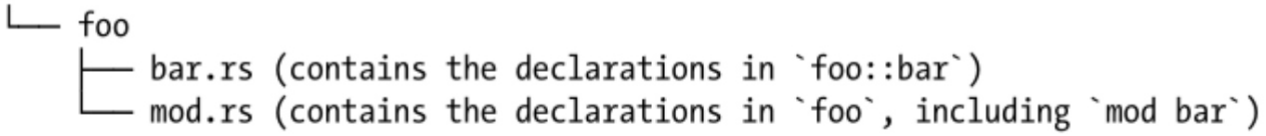
1. Make a new directory named 'network', the parent module's name.
2. Move the `src/network.rs` file into the new 'network' directory and rename it `src/network.rs/mod.rs`.
3. Move the submodule file `src/server.rs` into the network directory.

Therefore, in order to extract a file for the 'network::client' submodule of the 'network' module, we needed to create a directory for the 'network' module instead of a `src/network.rs` file.
The code that is in the network module then goes into the `src/network/mod.rs` file, and the submodule 'network::client' can have its own `src/network/client.rs` file.
Now the top-level `src/client.rs` is unambiguously the code that belongs to the client module.

Rules of Module filesystem

- If a module named 'foo' has no submodules, you should put the declarations for foo in a file named `foo.rs`.
- If a module named 'foo' does have submodules, you should put the declarations for foo in a file named `foo/mod.rs`.

This rules apply recursively, so if a module named 'foo' has a submodule named 'bar' and 'bar' does not have submodules, you should have the following files in your `/src`



The modules should be declared in their parent module's file using the mod keyword.

Controlling visibility with pub

The '/communicator' project we've built does compile with 'cargo build' but we still get warnings saying that the 'client::connect()', 'network::connect()' and 'network::server::connect()' functions are not being used

client::connect()', 'network::connect()' and 'network::server::connect()' functions are not being used.

In order to understand what's happeing we're going to create a new `/src/main.rs` file invoking the 'client::connect()' functions inside the main function.

```
                                                    error[E0603]: module 'client' is
private
```

The default state of all code in Rust is private: no one else is allowed to use the code.
If you don't use a private function within your program, because your program is the only code allowed to use that function, Rust will warn you that the function has gone unused.
After you specify that a function such as `client::connect()` is public, not only will your call to that function from your binary crate be allowed, but also the warning that the function is unused will go away.

Making a function public

To tell Rust to make a function public, we add the 'pub' keyword to the start of the declaration.

We can add this attribute both to modules and functions!

Summarizing privacy rules:

- If an item is public, it can be accessed through any of its parent modules.
- If an item is private, it can be accessed only by its immediate parent module and any of the parent's child modules.

Private items can be used only within the same crate, public ones expose themself to the outside-world.

Referring to names in different modules

```
pub mod a {
    pub mod series {
        pub mod of {
            pub fn nested_modules() {}
        }
    }
}

fn main() {
    a::series::of::nested_modules();
}
```

As you can see from this example, calling the 'nested_modules()' function can get lenghty when working on nested modules;
Rust provides a keyword to make it faster.

Bringing names into scope with the 'use' keyword

Rust's 'use' keyword shortens lengthy function calls by bringing the modules of the function you want to call into scope.
Here's an example:

```
pub mod a {
    pub mod series {
        pub mod of {
            pub fn nested_modules() {}
        }
    }
}
```

```
}

use a::series::of;

fn main() {
    of::nested_modules();
}
```

The ‘use’ keyword brings only what we’ve specified into scope;
it does not bring children of modules into scope.

That’s why we still have to use ‘of::nested_modules()’ when we want to call the ‘nested_modules()’ function.

We could also have bringed into scope only the function by specifing it:

```
pub mod a {
    pub mod series {
        pub mod of {
            pub fn nested_modules() {}
        }
    }
}

use a::series::of::nested_modules;

fn main() {
    nested_modules();
}
```

Because enums also form a sort of namespace like modules, we can bring an enum’s variants into scope with use as well.
For any kind of ‘use’ statement, if you’re bringing multiple items from one namespace into scope, you can list them using curly brackets and commas in the last position as follows:

```
enum TrafficLight {
    Red,
    Yellow,
    Green,
}

use TrafficLight::{Red, Yellow};

fn main() {
    let red = Red;
    let yellow = Yellow;
    let green = TrafficLight::Green;
}
```

We obviously need to specify the ‘TrafficLight’ namespace for the Green variant which we didn’t include into scope using the ‘use’ statement.

Bringing all names into scope with ‘glob operator’

The ‘*’ symbol is called ‘glob operator’.
We can bring all items in a namespace into scope at one using the glob operator as follows:

```
enum TrafficLight {
```

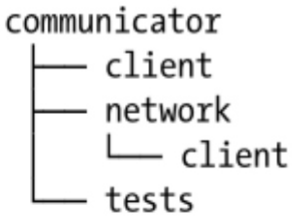
```
    Red,
    Yellow,
    Green,
}

use TrafficLight::*;

fn main() {
    let red = Red;
    let yellow = Yellow;
    let green = TrafficLight::Green;
}
```

Using 'super' to access a parent module

When you create a library crate, Cargo makes a 'tests' module for you as we saw in the `/communicator` example which shown in this hierarchy representation:



Tests are for exercising the code within our library, so let's try to call our 'client::connect()' function from this 'it_works()' function, even though we won't be checking any functionality right now.

```
                                                    error[E0433]: failed to
resolve. Use of undeclared type or module client
```

The reason is that paths are always relative to the current module, which here is 'tests'.

The only exception is in a 'use' statement, where paths are relative to the crate root by default;
our tests module needs the client module in its scope.

To solve it we can either use one of those:

- leading colons to start from the root

```
::client::connect();
```

or

- the 'super' to move up one module in the hierarchy from our current module

```
super::client::connect();
```

It would also be annoying to have to type 'super::' in each test, but we've already seen the tool for that solution: 'use'

The 'super::' functionality changes the path you give to use so it is relative to the parent module instead of to the root module.

```
#[cfg(test)]
mod tests {
    use super::client;
```

```
    use super::client;

    #[test]
    fn it_works() {
        client::connect();
    }
}
```

When we run ‘cargo test’ again, the test will pass!